AXI4-Lite Peripheral Subsystem — UVM Verification

SystemVerilog · UVM · SVA · Functional Coverage · Constrained Random

Ayman Shahapuri · Final-year ECE, Bengaluru · github.com/aym00n20005/axi4lite-uvm-verification

The design is deliberately small so the effort goes into the methodology. An address-decoding interconnect, a register file with mixed RW/RO/W1C/WC policies, and a byte-addressable memory. Everything interesting is in the machinery built to check them.

BUILT

- Specification frozen **before** any RTL; corrected before it, never after
- **23 SVA assertions + 14 cover properties**, bound to **three ports** — the interconnect checked as a slave from the master and as a master from each slave
- UVM agent with **five independent per-channel threads**, not a sequential FSM — a lockstep driver can never produce W-before-AW
- Passive monitor rebuilding transactions **from pins only**; scoreboard derived from the **specification**, and deliberately **not** generated from the same source as the design
- UVM RAL **generated** from one YAML alongside the RTL's register map and the docs, its mirror fed by the monitor rather than the stimulus
- A flow: regression runner, log parser with normalised **failure signatures**, triage, coverage trending, traceability matrix

MEASURED

Local regression	16 runs, **9,044 checks**, parallel, 12 seconds
Cover properties	**14 of 14** non-zero — five had never left zero
Functional coverage	11 groups + 4 crosses, all modelled, sampled from observation
Traceability	**29 features** → test → result, generated from the spec's own list
Formal	16 properties, depth 25 and 40, with a cover pass proving none is vacuous
Bugs	**6 of 6** injections caught by their assigned mechanism · **4** genuine, all in the testbench
LLM triage	**4/6 correct, 2/6 partial, 0/6 wrong** — sealed from the source

TOOLS

Cadence Xcelium (UVM) · **slang** (local elaboration of the whole UVM environment, 0.2 s) · Verilator (RTL & SVA) · **Yosys + Z3** (bounded model check) · Make + Python · Git. Six tools, no two of which do the same job — measured, not assumed.

WHAT THE TESTBENCH FOUND

> **A rule that 4,576 simulation checks cannot decide.**
> The spec requires that a hardware event colliding with a write-1-to-clear on the same cycle wins. I inverted that priority in the RTL. Three directed benches, five random seeds, **4,576 checks** and 23 bound assertions **all passed**; the bounded model check failed in under a second, naming the property. The collision happens inside one transaction and the scoreboard compares across whole transactions — no number of seeds reaches it.

> **Two constraints that meant something other than what they said.**
> `soft (a) -> b dist {...}` selects the wrong production of the IEEE 1800-2017 constraint grammar. Xcelium accepts it, and every number this project had published came through it. Found in **0.2 seconds** by pointing a second, conformant front end at the existing sources. A testbench validated by one simulator is validated against that simulator's extensions.

> **A cover point that could not fail to be covered.**
> The property meant to prove the driver's threads run independently was satisfied by *any* multi-transaction run. It could never stay uncovered, so it could never warn. Qualifying it dropped the hit count from **10 to 1** on identical stimulus.

Honest scope. Two features are reachable only under UVM, which runs on one commercial simulator reached through a browser. Some cover points are zero at the slave ports and **must stay zero** — `DECERR` there would be a defect, not coverage. The formal proof is bounded. Code coverage was not done, for a measured reason: the only simulator that can produce it is reached through a fixed command line that cannot enable it. Every number above came from a run; nothing is projected.

Verification plan, DUT spec, 10 bug reports, formal notes and a full walkthrough in `docs/` make all